

果有的话)。

```
(a) int calc(int&, int&);
    int calc(const int&, const int&);
(b) int calc(char*, char*);
    int calc(const char*, const char*);
(c) int calc(char*, char*);
    int calc(char* const, char* const);
```



6.7 函数指针

函数指针指向的是函数而非对象。和其他指针一样，函数指针指向某种特定类型。函数的类型由它的返回类型和形参类型共同决定，与函数名无关。例如：

```
// 比较两个 string 对象的长度
bool lengthCompare(const string &, const string &);
```

该函数的类型是 `bool(const string&, const string&)`。要想声明一个可以指向该函数的指针，只需要用指针替换函数名即可：

```
// pf 指向一个函数，该函数的参数是两个 const string 的引用，返回值是 bool 类型
bool (*pf)(const string &, const string &); // 未初始化
```

从我们声明的名字开始观察，pf 前面有个*，因此 pf 是指针；右侧是形参列表，表示 pf 指向的是函数；再观察左侧，发现函数的返回类型是布尔值。因此，pf 就是一个指向函数的指针，其中该函数的参数是两个 const string 的引用，返回值是 bool 类型。



*pf 两端的括号必不可少。如果不写这对括号，则 pf 是一个返回值为 bool 指针的函数：

```
// 声明一个名为 pf 的函数，该函数返回 bool*
bool *pf(const string &, const string &);
```

◀ 248

使用函数指针

当我们把函数名作为一个值使用时，该函数自动地转换成指针。例如，按照如下形式我们可以将 lengthCompare 的地址赋给 pf：

```
pf = lengthCompare;           // pf 指向名为 lengthCompare 的函数
pf = &lengthCompare;         // 等价的赋值语句：取地址符是可选的
```

此外，我们还能直接使用指向函数的指针调用该函数，无须提前解引用指针：

```
bool b1 = pf("hello", "goodbye");           // 调用 lengthCompare 函数
bool b2 = (*pf)("hello", "goodbye");         // 一个等价的调用
bool b3 = lengthCompare("hello", "goodbye"); // 另一个等价的调用
```

在指向不同函数类型的指针间不存在转换规则。但是和往常一样，我们可以为函数指针赋一个 nullptr（参见 2.3.2 节，第 48 页）或者值为 0 的整型常量表达式，表示该指针没有指向任何一个函数：

```
string::size_type sumLength(const string&, const string&);
bool cstringCompare(const char*, const char*);
pf = 0;                      // 正确：pf 不指向任何函数
pf = sumLength;               // 错误：返回类型不匹配
```

```
pf = cstringCompare;    // 错误：形参类型不匹配
pf = lengthCompare;    // 正确：函数和指针的类型精确匹配
```

重载函数的指针

当我们使用重载函数时，上下文必须清晰地界定到底应该选用哪个函数。如果定义了指向重载函数的指针

```
void ff(int*);
void ff(unsigned int);

void (*pf1)(unsigned int) = ff; // pf1 指向 ff(unsigned)
```

编译器通过指针类型决定选用哪个函数，指针类型必须与重载函数中的某一个精确匹配

```
void (*pf2)(int) = ff;    // 错误：没有任何一个 ff 与该形参列表匹配
double (*pf3)(int*) = ff; // 错误：ff 和 pf3 的返回类型不匹配
```



249

函数指针形参

和数组类似（参见 6.2.4 节，第 193 页），虽然不能定义函数类型的形参，但是形参可以是指向函数的指针。此时，形参看起来是函数类型，实际上却是当成指针使用：

```
// 第三个形参是函数类型，它会自动地转换成指向函数的指针
void useBigger(const string &s1, const string &s2,
               bool pf(const string &, const string &));
// 等价的声明：显式地将形参定义成指向函数的指针
void useBigger(const string &s1, const string &s2,
               bool (*pf)(const string &, const string &));
```

我们可以直接把函数作为实参使用，此时它会自动转换成指针：

```
// 自动将函数 lengthCompare 转换成指向该函数的指针
useBigger(s1, s2, lengthCompare);
```

正如 useBigger 的声明语句所示，直接使用函数指针类型显得冗长而烦琐。类型别名（参见 2.5.1 节，第 60 页）和 decltype（参见 2.5.3 节，第 62 页）能让我们简化使用了函数指针的代码：

```
// Func 和 Func2 是函数类型
typedef bool Func(const string&, const string&);
typedef decltype(lengthCompare) Func2;    // 等价的类型
// FuncP 和 FuncP2 是指向函数的指针
typedef bool(*FuncP)(const string&, const string&);
typedef decltype(lengthCompare) *FuncP2;  // 等价的类型
```

我们使用 typedef 定义自己的类型。Func 和 Func2 是函数类型，而 FuncP 和 FuncP2 是指针类型。需要注意的是，decltype 返回函数类型，此时不会将函数类型自动转换成指针类型。因为 decltype 的结果是函数类型，所以只有在结果前面加上*才能得到指针。可以使用如下的形式重新声明 useBigger：

```
// useBigger 的等价声明，其中使用了类型别名
void useBigger(const string&, const string&, Func);
void useBigger(const string&, const string&, FuncP2);
```

这两个声明语句声明的是同一个函数，在第一条语句中，编译器自动地将 `Func` 表示的函数类型转换成指针。

返回指向函数的指针

和数组类似（参见 6.3.3 节，第 205 页），虽然不能返回一个函数，但是能返回指向函数类型的指针。然而，我们必须把返回类型写成指针形式，编译器不会自动地将函数返回类型当成对应的指针类型处理。与往常一样，要想声明一个返回函数指针的函数，最简单的办法是使用类型别名：

```
using F = int(int*, int);           // F 是函数类型，不是指针
using PF = int(*) (int*, int);     // PF 是指针类型
```

其中我们使用类型别名（参见 2.5.1 节，第 60 页）将 `F` 定义成函数类型，将 `PF` 定义成指向函数类型的指针。必须时刻注意的是，和函数类型的形参不一样，返回类型不会自动地转换成指针。我们必须显式地将返回类型指定为指针：

◀ 250

```
PF f1(int);           // 正确：PF 是指向函数的指针，f1 返回指向函数的指针
F f1(int);            // 错误：F 是函数类型，f1 不能返回一个函数
F *f1(int);           // 正确：显式地指定返回类型是指向函数的指针
```

当然，我们也能用下面的形式直接声明 `f1`：

```
int (*f1(int))(int*, int);
```

按照由内向外的顺序阅读这条声明语句：我们看到 `f1` 有形参列表，所以 `f1` 是个函数；`f1` 前面有 `*`，所以 `f1` 返回一个指针；进一步观察发现，指针的类型本身也包含形参列表，因此指针指向函数，该函数的返回类型是 `int`。

出于完整性的考虑，有必要提醒读者我们还可以使用尾置返回类型的方式（参见 6.3.3 节，第 206 页）声明一个返回函数指针的函数：

```
auto f1(int) -> int (*) (int*, int);
```

将 `auto` 和 `decltype` 用于函数指针类型

如果我们明确知道返回的函数是哪一个，就能使用 `decltype` 简化书写函数指针返回类型的过程。例如假定有两个函数，它们的返回类型都是 `string::size_type`，并且各有两个 `const string&` 类型的形参，此时我们可以编写第三个函数，它接受一个 `string` 类型的参数，返回一个指针，该指针指向前两个函数中的一个：

```
string::size_type sumLength(const string&, const string&);
string::size_type largerLength(const string&, const string&);
// 根据其形参的取值，getFcn 函数返回指向 sumLength 或者 largerLength 的指针
decltype(sumLength) *getFcn(const string &);
```

声明 `getFcn` 唯一需要注意的地方是，牢记当我们将 `decltype` 作用于某个函数时，它返回函数类型而非指针类型。因此，我们显式地加上 `*` 以表明我们需要返回指针，而非函数本身。

6.7 节练习

练习 6.54：编写函数的声明，令其接受两个 `int` 形参并且返回类型也是 `int`；然后声明一个 `vector` 对象，令其元素是指向该函数的指针。

练习 6.55: 编写 4 个函数，分别对两个 `int` 值执行加、减、乘、除运算；在上一题创建的 `vector` 对象中保存指向这些函数的指针。

练习 6.56: 调用上述 `vector` 对象中的每个元素并输出其结果。

小结

251

函数是命名了的计算单元，它对程序（哪怕是不大的程序）的结构化至关重要。每个函数都包含返回类型、名字、（可能为空的）形参列表以及函数体。函数体是一个块，当函数被调用的时候执行该块的内容。此时，传递给函数的实参类型必须与对应的形参类型相容。

在 C++ 语言中，函数可以被重载：同一个名字可用于定义多个函数，只要这些函数的形参数量或形参类型不同就行。根据调用时所使用的实参，编译器可以自动地选定被调用的函数。从一组重载函数中选取最佳函数的过程称为函数匹配。

术语表

二义性调用 (ambiguous call) 是一种编译时发生的错误，造成二义性调用的原因是在函数匹配时两个或多个函数提供的匹配一样好，编译器找不到唯一的最佳匹配。

实参 (argument) 函数调用时提供的值，用于初始化函数的形参。

Assert 是一个预处理宏，作用于一条表示条件的表达式。当未定义预处理变量 `NDEBUG` 时，`assert` 对条件求值。如果条件为假，输出一条错误信息并终止当前程序的执行。

自动对象 (automatic object) 仅存在于函数执行过程中的对象。当程序的控制流经过此类对象的定义语句时，创建该对象；当到达了定义所在的块的末尾时，销毁该对象。

最佳匹配 (best match) 从一组重载函数中为调用选出的一个函数。如果存在最佳匹配，则选出的函数与其他所有可行函数相比，至少在一个实参上是更优的匹配，同时在其他实参的匹配上不会更差。

传引用调用 (call by reference) 参见引用传递。

传值调用 (call by value) 参见值传递。

候选函数 (candidate function) 解析某次函数调用时考虑的一组函数。候选函数的名字应该与函数调用使用的名字一致，并且在调用点候选函数的声明在作用域之内。

constexpr 可以返回常量表达式的函数，一个 `constexpr` 函数被隐式地声明成内联函数。

默认实参 (default argument) 当调用缺少了某个实参时，为该实参指定的默认值。

可执行文件 (executable file) 是操作系统能够执行的文件，包含着与程序有关的代码。

函数 (function) 可调用的计算单元。

函数体 (function body) 是一个块，用于定义函数所执行的操作。

函数匹配 (function matching) 编译器解析重载函数调用的过程，在此过程中，实参与每个重载函数的形参列表逐一比较。

函数原型 (function prototype) 函数的声明，包含函数名字、返回类型和形参类型。要想调用某函数，在调用点之前必须声明该函数的原型。

隐藏名字 (hidden name) 某个作用域内声明的名字会隐藏掉外层作用域中声明的同名实体。

initializer_list 是一个标准类，表示的是一组花括号包围的类型相同的对象，对象之间以逗号隔开。

内联函数 (inline function) 请求编译器在可能的情况下在调用点展开函数。内联函数可以避免常见的函数调用开销。

链接 (link) 是一个编译过程，负责把若干

252